IEEESTEC – 19th Student Projects Conference, Niš, 2026.
https://doi.org/organizer will add number later
Original scientific paper

Fiducial: Build Any Robust Multidisciplinary Product with AI Agents

Aleksa Zdravković
12th grade high school student
ETŠ "Nikola Tesla" Niš
Niš, Serbia
aleksazdravkovic@proton.me
ORCID: 0009-0004-1535-1839

***Abstract*—This paper presents Fiducial, an open-source platform for building products that span software and hardware with AI coding agents. Agents now write each part of a product quickly, but the facts the parts share are still copied between them and drift apart. Fiducial declares every fact once, generates every artifact from it – code, translations, circuit board and enclosure – and checks every change, so agents work within guarantees instead of being trusted. In a test of 20 realistic mistakes, the checks caught all 20 before they could be merged.**

Keywords—AI agents, single source of truth, code generation, consistency checking, hardware–software co-design, continuous integration

I. Introduction

Building a product used to take a team: someone for the circuit board, someone for the firmware, others for the app, the website and the enclosure. AI coding agents such as Claude Code [1] and Codex [2] are changing that. An agent can write a firmware driver, a web page, a database migration or a script that models a case in minutes, so one person can now attempt a whole product.

Building such products – first Outreach [3], an off-grid messaging system made of radios, gateways, an app and printed cases, and then the platform described here – has shown where the difficulty has moved. Writing each part is no longer the slow step; keeping the parts in agreement is. In Outreach, the radio's firmware and the app disagreed on the layout of a message, and a printed case did not fit the board it was made for. Neither part was wrong on its own. The facts they shared had been copied by hand from one tool to another, one copy changed, and the mistake appeared only when the parts met (Fig. 1a).

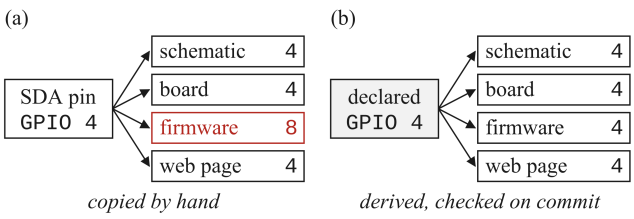


Fig. 1. One fact used in four places: (a) copied by hand, one copy changes and nothing notices; (b) declared once, every copy is generated from it and checked.

Agents do not remove this problem; they speed it up. An agent sees only the codebase it works in, so the agent writing the firmware does not know the board changed, and the one writing the web page does not know the firmware did. Its output is also probabilistic: usually right, but never guaranteed. More code, written faster, in more disconnected places, gives an inconsistent product sooner.

What is missing is not a better tool for any one discipline, but one shared and checked context for all of them: a single place where a product's facts live, from which every part is produced, and which refuses a change that makes the parts disagree (Fig. 1b). This paper presents Fiducial [4], an open-source platform built to provide it, compares it with existing approaches, and tests whether its checks catch the mistakes that people and agents actually make.

II. Existing Approaches

Several kinds of tool already address parts of this problem, and each falls short in the same way: it keeps one kind of fact, or one pair of disciplines, in agreement, but not the product as a whole (Table I).

Model-driven and model-based systems engineering keep a design in one central model and generate or check other views from it [5], [6]. They are the closest in spirit, but the model lives in a specialised graphical tool, the code is still written by hand beside it, and an agent cannot easily work in it. Engineering suites link neighbouring disciplines instead: Altium's MCAD CoDesigner passes changes between a circuit board and its mechanical model [7]. The firmware, the app and the translations stay outside, and the link is between two tools, not from one source.

Design-as-code tools write hardware as text: a circuit in atopile [8], a case as a build123d script [9]. This is what makes hardware writable by an agent, and Fiducial uses build123d. But each covers one discipline. Fiducial's first design planned to author boards in atopile, and found that a circuit language does not cover the case, the layout or the cost, so the board's facts would still be copied into each of them. Interface languages such as Protocol Buffers declare a message once and generate the code for both sides [10]. That removes the drift met in Outreach, but only for messages; a pin, a colour or a board size has no such home.

TABLE I. EXISTING APPROACHES AND WHERE THEY STOP

Approach	Keeps in agreement	Stops at
Model-based engineering	views of one model	graphical tool; code by hand
ECAD–MCAD links	board and case	two disciplines, two tools
Design as code	one discipline	facts shared with others
Interface languages	messages, both sides	everything else
Build systems	outputs with files	two files that disagree
Agent context files	nothing; they advise	not enforced
Fiducial	every declared fact	facts not yet declared

Build systems such as Bazel [11] and Nix [12] rebuild exactly what changed, and Nix identifies every result by a fingerprint of its inputs. They guarantee that outputs follow from files, not that files agree with each other: the same fact typed into two source files builds without complaint. Coding agents, finally, follow context files such as AGENTS.md [13], which describe a repository's rules but do not enforce them. A known way to use a generator that can be wrong is to pair it with a checker that cannot [14], but no existing checker spans a whole product.

Fiducial keeps what works in each – text that agents can write, outputs tied to inputs by fingerprints, established engines for hard problems, and checks that decide – and changes the unit. It tracks facts rather than files or tools, across every discipline of one product, so any fact – a message, a pin, a colour, a size – has exactly one home.

III. THE APPROACH

Fiducial treats a product as two kinds of thing. Facts are the decisions that define it: which pin a sensor is connected to, what a button says in each language, the brand colour, the size of the board, the layout of a message between a device and its app. Artifacts are everything built from those facts: source code, schematics, circuit boards, translation files, web pages, enclosure models. Drift is what happens when an artifact stops agreeing with a fact, or with another artifact that uses the same fact (Fig. 1a). The approach consists of three rules that together make drift visible and keep it out of the product (Fig. 1b).

Declare once. Every fact is written in exactly one place, a declaration: a typed text file in the product's repository. Typed means the file has a fixed structure that a program can check, not free prose. A fact that exists once has no second copy to disagree with.

Derive everything else. Every artifact that depends on a fact is generated from its declaration by one command, and generated files are never edited by hand. Generation is deterministic: the same declarations always produce exactly the same files, which makes the product reproducible and makes any difference meaningful.

Check every change. Before a change is accepted, checks confirm that every artifact still matches what its declarations produce now, and that the declarations themselves are valid. A failing check names the declaration and the file at fault, so the person or agent knows what to fix.

These rules change the role of an AI agent. The agent still edits declarations and code, but it is no longer trusted to keep anything consistent: generation does that, and the checks confirm it. Its mistakes are not prevented, but they are caught before they reach the product, with a message precise enough for the agent to correct itself. Hence the name: a fiducial is a fixed reference point, a universal measure that everything is aligned to and compared against, so it stays uniform. The declarations are that reference for a product.

IV. IMPLEMENTATION

Fiducial is a command-line tool, fid, written in Rust. Its parts are described below; Fig. 2 shows how a change moves through them.

A. One Structure for Every Product

fid new creates a product repository with a fixed layout, and fid add enables capabilities. A capability packages one discipline: its declarations, the generator that turns them into artifacts, its checks and instructions for agents. There are 23, from web applications and translations to firmware and hardware. Because every product has the same structure, whoever knows one knows them all, and a new product starts from parts that already work. Adding translations, for example, brings the language files, their generator, the check that every language has every string, and the agent's instructions. fid upgrade brings platform improvements into each product while keeping its own edits.

B. Declarations and Generation

Declarations are plain text files in version control, from the strings of each language to every part of a board with the net each pin connects to. Text was chosen over a graphical modelling tool because coding agents read and write it well, and every change then goes through ordinary code review. fid derive runs each capability's generator, which reads the declarations and writes the artifacts, from a typed module of translations to a firmware pin map or a KiCad circuit board [15]. A lock file records a cryptographic hash – a short fingerprint of a file's contents – for every input and output, so the platform knows exactly what each artifact was made from.

C. Checks

Checks run in continuous integration, the automatic build that runs on every proposed change, and a change is merged only when they pass. There are three kinds. Freshness: fid derive --check compares every recorded fingerprint with the repository and fails if a declaration changed without its artifacts being regenerated, or if a generated file was edited by hand. Rules: generation refuses declarations that break a rule, such as a language missing a string, two colours with too little contrast, parts that cost more than the declared limit, or a board that does not fit its case. Consumers: compilers and type checkers read the generated files, so code that still uses a renamed fact stops compiling.

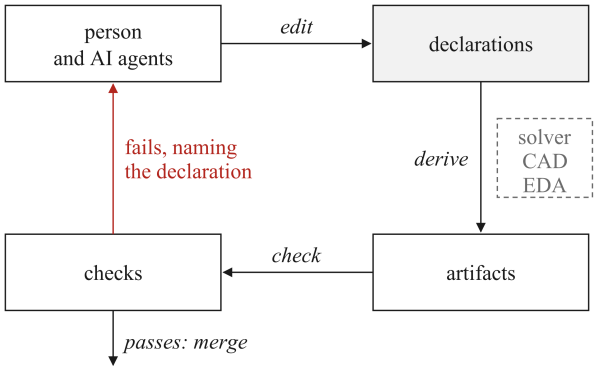


Fig. 2. How a change moves through Fiducial: declarations are edited, generation produces the artifacts with the help of established engines, and the checks either accept the change or return a message naming the declaration at fault.

D. Engines for Hard Problems

Some generation steps are real engineering problems: where to place the parts on a board, how to connect them, how to build a case around them. The first version placed parts with a hand-written search, and each new product added special rules until they began to interact. Now the platform only writes the problem down and hands it to an established engine. A constraint solver, CP-SAT [16], is given the rules a placement must satisfy and finds one that does, or proves none exists; build123d [9] builds the case; KiCad routes and checks the board. Every placement rule is one of eight general kinds, such as inside, apart or near, so a new product needs new declarations rather than new code. The solver's answer is stored with the fingerprint of the problem it solved, so checking a product never runs it again. The engines check as well as build: KiCad confirms the board can be made, and the case model confirms that the board fits and the lid closes.

E. Agents in the Loop

Every product carries an AGENTS.md file and one instruction file per capability, so any agent starts from the same description of the product, and a hook stops it from editing a generated file. fid dash shows the whole product's state in one view, recomputed from the repository, so an agent starts from facts rather than guesses. Agents fix a problem where the fact lives, never in the output. The platform grows the same way: a mistake that gets past the checks becomes a general rule for every product.

V. Example: A Sensor Stick

The repository contains a complete example product: a USB-C stick that measures temperature and humidity and shows them on a web page (Fig. 3). It is small, but it has every discipline: a printed case, a circuit board, firmware in Rust and a page that reads the sensor in the browser. It was chosen as the smallest product that still crosses all of them, so every rule described above applies to it.

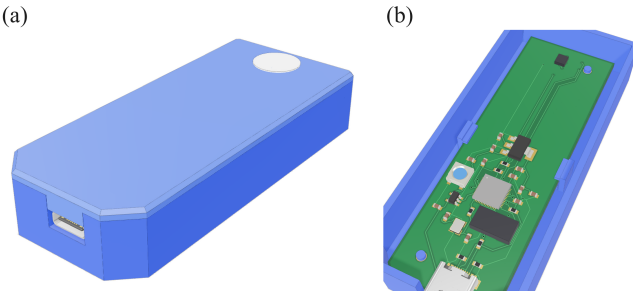


Fig. 3. The sensor stick, generated from its declaration: (a) the closed case; (b) the routed board inside it, with the sensor under its vent.

The person writes the product declaration, an outline of its shape, the firmware's logic and the page. Everything else is generated: the routed board with its schematic and bill of materials, the case built around it, the files a manufacturer needs, and the code the firmware and the page use to exchange readings. Moving the sensor to another pin is one changed line. Until the product is regenerated the check fails; afterwards the board, the firmware's pin map and the page's types have all followed, with no edit to the firmware or the page. If the new pin is one the chip's sensor bus cannot use, the firmware stops compiling – before any board is made.

VI. Evaluation

Fiducial was built over six weeks by one person working with AI agents, through 68 reviewed pull requests, and used on two products: the sensor stick and a private product. On the sensor stick, moving one pin regenerates 8 files in 37 s.

Checks are only useful if they catch the mistakes that people and agents actually make. The test borrows the idea of mutation testing [17]: a realistic mistake is introduced on purpose, the checks are run, and the result shows whether they noticed. Twenty mistakes cover eight kinds of drift met while building Fiducial and its products (Table II): ten in a fresh web product in English and Serbian, and ten in the sensor stick's hardware, firmware and the interface between them. Each was applied to a clean copy, the checks ran as in continuous integration, and the copy was reset. Three correct changes and two runs with no change tested the opposite direction: that the checks do not fail without a reason. The test is a script in the repository, and every failure names the file and the declaration at fault (Fig. 4).

On release 0.9.4 the checks caught 15 of the 20, and each of the five misses was a fact the platform did not yet own. Two were declarations edited without regenerating: only the hardware generator recorded which files it read, so the freshness check did not notice that the others' inputs had changed. In two, the firmware wrote a pin number or a sensor address directly instead of taking it from the declaration. The last was a Serbian string left in English, complete in structure but not translated.

TABLE II. INJECTED MISTAKES, BY KIND OF DRIFT

Kind of drift	Example	0.9.4	0.9.11
Stale output	Domain edited, not re-derived	1/3	3/3
Generated file edited	Pin map edited by hand	5/5	5/5
Missing declaration	String in English only	3/3	3/3
Rule broken	Contrast below 4.5 : 1	3/3	3/3
Declarations conflict	Case too narrow for board	2/2	2/2
Code out of date	Net renamed, old name used	1/1	1/1
Declaration bypassed	Pin typed as a number	0/2	2/2
Not translated	Serbian left in English	0/1	1/1
Total		15/20	20/20

(a)

```
$ fid derive
fid-i18n: the catalogs are not complete.
messages/sr.json is missing 1 key(s):
action.delete
A missing translation is a missing artifact,
not a fallback (MISSION.md 1c).
```

(b)

```
$ fid derive
firmware types what the board already declares
(a pin, an address, a command or a wait):
firmware/rp2040/src/main.rs:84: takes PIN_4
directly — take it through
`board::<net>!(p)`
(hardware/generated/board.rs) so a moved pin
moves it too, …
```

Fig. 4. Two failures as the person or agent sees them: (a) a string added in English only; (b) firmware naming a pin directly instead of taking it from the declaration.

Each miss became a general rule: every generator now records its inputs, firmware that writes a declared value directly is refused, and a translation identical to the default language fails unless it is declared the same on purpose. On release 0.9.11, all 20 are caught – 10 during generation, 9 by the freshness check and 1 by the compiler – and every correct change passes.

The mistakes were chosen by the author of the platform, so they may favour what it was built to catch. Only two products have been built, and neither has yet been manufactured and tested. The test measures the checks, not whether agents working with Fiducial make fewer mistakes than agents working without it.

VII. CONCLUSION

AI agents have made writing each part of a product fast, and that has moved the hard problem to keeping the parts in agreement. Better models will move the line between what a person checks and what is left to agents, but a probabilistic system cannot guarantee its output, and even a deterministic tool is only right about what it was told. Fiducial answers this at the level of the product rather than the codebase. In practice, it let one person working with AI agents keep a product's firmware, web page, circuit board and case in agreement, with every injected mistake caught early by a message that names its cause. The next steps are to manufacture the sensor stick and to compare agents working with and without the platform. Fiducial and its example are open source [4].

REFERENCES

[1] Claude Code documentation. Available at: https://docs.anthropic.com/en/docs/claude-code

[2] OpenAI Codex. Available at: https://openai.com/codex/

[3] A. Zdravković, Đ. Jovanović and P. Šubarević, "Outreach: Modular AI-enhanced off-grid mesh communication system for remote environments," IEEESTEC – 18th Student Projects Conference, Niš, 2025.

[4] A. Zdravković, Fiducial, release 0.9.11, 2026. Available at: https://github.com/AleksaZCodes/fiducial

[5] D. C. Schmidt, "Guest editor's introduction: Model-driven engineering," Computer, vol. 39, no. 2, pp. 25–31, 2006.

[6] S. J. I. Herzig, A. Qamar and C. J. J. Paredis, "An approach to identifying inconsistencies in model-based systems engineering," Procedia Computer Science, vol. 28, pp. 354–362, 2014.

[7] Altium, MCAD CoDesigner, ECAD–MCAD collaboration. Available at: https://www.altium.com/

[8] atopile, a language for designing circuit boards as code. Available at: https://github.com/atopile/atopile

[9] build123d, a Python CAD library on the OpenCascade kernel. Available at: https://github.com/gumyr/build123d

[10] Protocol Buffers. Available at: https://protobuf.dev/

[11] Bazel, a build and test tool. Available at: https://bazel.build/

[12] E. Dolstra, M. de Jonge and E. Visser, "Nix: A safe and policy-free system for software deployment," in Proc. LISA, 2004, pp. 79–92.

[13] AGENTS.md, a format for guiding coding agents. Available at: https://agents.md/

[14] S. Kambhampati et al., "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in Proc. ICML, PMLR 235, 2024.

[15] KiCad EDA. Available at: https://www.kicad.org/

[16] L. Perron and F. Didier, CP-SAT, Google OR-Tools. Available at: https://developers.google.com/optimization/cp/cp_solver

[17] R. A. DeMillo, R. J. Lipton and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," Computer, vol. 11, no. 4, pp. 34–41, 1978.